SC-LAB-004

Análisis de Seguridad Estático y Dinámico
SAST + DAST

Manual del Alumno

SecureCampus Security Lab · versión 1.0 · septiembre 2026

Pregunta guía
¿Qué puede descubrir una herramienta al analizar el código fuente y qué puede descubrir al interactuar con una aplicación en ejecución?

1. Propósito y resultados de aprendizaje

Analizar una miniaplicación de SecureCampus (Spring Boot) con técnicas SAST y DAST, interpretar evidencia, corregir dos vulnerabilidades deliberadas y comprobar las correcciones mediante reanálisis.

- Distinguir SAST de DAST por objeto de análisis, momento de ejecución y evidencia.
- Ejecutar Semgrep sobre código Java/Spring Boot y OWASP ZAP sobre el servidor local de SecureCampus.
- Interpretar hallazgos sin asumir que 0 findings, PASS o WARN equivalen a una garantía absoluta.
- Corregir la persistencia y validación de relaciones en entidades JPA (como `Estudiante` y `Profesor` en `AdminService.java`).
- Validar y corregir controles de acceso y renderizado seguro en vistas Thymeleaf (`admin/dashboard.html`).
- Registrar trazabilidad técnica con Git.

2. Requisitos del laboratorio

Componente	Requisito
Sistema	Windows /Linux + PowerShell + VS Code
Git	Repositorio loca `SecureCampus-SecurityLab / SecureCampus`
Python	Java 17+, maven, Mysql
Web	Spring Boot
Docker	Docker Desktop con Engine activo
SAST	Semgrep Community mediante contenedor
DAST	OWASP ZAP mediante contenedor

Regla de seguridad
Todos los escaneos activos de esta práctica se realizan únicamente contra la aplicación local deliberadamente vulnerable del laboratorio. No se deben dirigir contra sistemas externos o sin autorización.

3. Preparación

```
cd C:\Users\nombreUsuario\Documents\SecureCampus
git status
```

La estructura del proyecto incluye `src/main/java/com/ds/securecampus/`, `pom.xml`, `Docs/security/`, entre otros

4. Parte A · SAST con Semgrep

4.1 Analiza primero como desarrollador

Revisa los archivos del proyecto, específicamente `AdminService.java` y `AdminController.java`. Identifica el flujo de datos desde los modales de registro hasta los repositorios JPA.

Pregunta	Respuesta del equipo
¿Qué dato controla el usuario?	Los parámetros de entrada enviados desde el modal de registro institucional (número de control, correo, rol y adscripción/carrera).
¿A dónde llega ese dato?	Al servicio `AdminService.java` y a los repositorios de Spring Data (`UsuarioRepository`, `EstudianteRepository`, `ProfesorRepository`)
¿Qué riesgo observas?	Excepciones de aserción de Hibernate por identificadores nulos (`null identifier`) si las entidades relacionadas con `@MapsId` no se enlazan correctamente antes del guardado.
¿Qué control propondrías?	Validar la integridad de los identificadores, normalizar textos de roles a mayúsculas y utilizar métodos de persistencia síncrona como `saveAndFlush`.

4.2 Ejecuta Semgrep

Registra el hallazgo principal en relación con la estructura de servicios de Spring Boot.

Evidencia SAST	Registro
Regla/hallazgo	Análisis estático de dependencias y flujos de persistencia en ORM (revisar cómo la aplicación maneja el guardado, la actualización y las relaciones entre esas tablas y clases)
Archivo/línea	src/main/java/com/ds/securecampus/service/AdminService.java
¿Qué evidencia aporta?	Detecta puntos donde se intenta fusionar entidades dependientes (`Estudiante`/`Profesor`) en el contexto de persistencia sin su clave primaria mapeada.
¿Coincide con tu análisis humano?	Sí, explica los errores de identificador nulo experimentados al dar de alta cuentas institucionales.

4.3 Corrige y reanaliza

- **Análisis de Consultas:** Se verifica que EstudianteRepository utilice exclusivamente métodos derivados de Spring Data JPA o JPQL parametrizado, evitando el uso de consultas nativas mal estructuradas que expongan al sistema a Inyección SQL.

CODIGO 1

```
package com.ds.securecampus.repository;



import com.ds.securecampus.model.Estudiante;

import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.stereotype.Repository;

import java.util.List;
```

```
@Repository

public interface EstudianteRepository extends JpaRepository<Estudiante, Long> {



    // Método derivado gestionado automáticamente por Spring Data JPA

    List<Estudiante> findByNombreContainingIgnoreCase(String nombre);



}
```

1. Source : Los parámetros enviados por el usuario a través de los formularios o peticiones HTTP llegan a los controladores y se pasan a los servicios y repositorios.
2. Flow : Spring Data JPA intercepta la llamada al método findByNombreContainingIgnoreCase y traduce de forma automática la invocación a una sentencia parametrizada segura en el ORM (Hibernate).
3. Sink : La ejecución se realiza en la base de datos TiDB utilizando prepared statements, separando completamente la estructura SQL de los datos proporcionados por el usuario.

- **Riesgo Identificado en Implementaciones Inseguras:** Si se hubieran construido consultas mediante concatenación de cadenas en consultas nativas, el sistema sería vulnerable.
- **Mitigación Aplicada:** El uso nativo de Spring Data JPA y JPQL parametrizado neutraliza cualquier intento de alteración maliciosa de la estructura de la consulta

CODIGO 2

- **Control de Acceso (Seguridad):** En SecurityConfig.java, se asegura la definición correcta de los roles en mayúsculas (hasRole("ADMIN"), hasRole("PROFESOR"), hasRole("ESTUDIANTE")) tal como lo exigen las normativas de Spring Security 6.x para evitar fallos lógicos en los filtros de autorización.
-

```
@Configuration @EnableWebSecurity public class SecurityConfig {

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable()) // Configuración evaluada en entorno de laboratorio
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/", "/login", "/css/**", "/images/**").permitAll()
            .requestMatchers("/admin/**").hasRole("ADMIN")
            .requestMatchers("/profesor/**").hasRole("PROFESOR")
            .requestMatchers("/estudiante/**").hasRole("ESTUDIANTE")
            .anyRequest().authenticated()
        )
        .formLogin(form -> form
            .loginPage("/login")
            .successHandler(new CustomAuthenticationSuccessHandler())
```

```
            .failureHandler(new CustomAuthenticationFailureHandler())
            .permitAll()
        );
    return http.build();
}


}
```

1. Control de Rutas: Define explícitamente qué endpoints son públicos (/login, recursos estáticos) y cuáles requieren roles específicos (ADMIN, PROFESOR, ESTUDIANTE).
2. Manejo de Autenticación: Vincula manejadores personalizados (CustomAuthenticationSuccessHandler y CustomAuthenticationFailureHandler) para auditar los intentos de acceso y evitar la fuga de información sensible en los mensajes de error.
3. Persistencia con MySQL: Este esquema de seguridad protege las rutas que posteriormente consultan la base de datos alojada en la nube mediante Spring Data JPA, garantizando que operen bajo un contexto autenticado.

- **Evaluación SAST:** Las herramientas de análisis estático verifican que los roles estén debidamente mapeados y que no existan rutas sensibles expuestas sin validación de sesión.

5. Parte B · DAST con OWASP ZAP

5.1 Inicia la aplicación

Ejecuta el proyecto Spring Boot localmente en VS Code (asegurando que corra en http://localhost:8080).

```
.\mvnw spring-boot:run
```

5.2 Baseline: observación pasiva

```
http://localhost:8080
```

Registra los WARN principales en el contexto de tu portal Spring Boot:

Hallazgo baseline	¿Qué observa?	¿Requiere análisis?
Missing Anti-clickjacking Header	Ausencia de la cabecera X-Frame-Options en el entorno Spring Boot.	Sí
X-Content-Type-Options Header Missing	Falta de la directiva `nosniff` en recursos estáticos (`portal.css`).	Sí
CSP Header Not Set	Ausencia de una política estricta de contenidos (Content-Security-Policy).	Sí
Otro:	Exposición de metadatos del servidor embebido (Tomcat).	Si

5.3 Active Scan: interacción deliberada

```
docker run --rm -t ghcr.io/zaproxy/zaproxy:stable zap-full-scan.py -t http://host.docker.internal:8080
```

Inspecciona los endpoints del panel de control central, gestión de identidades y bitácora de auditoría en AdminController.java. 2

5.4 Validación manual del XSS

Se introducen cadenas y caracteres especiales en los campos de entrada de la interfaz web para verificar que la aplicación procese los datos de forma codificada y segura, previniendo la ejecución de scripts arbitrarios en el navegador.

5.5 Registra la línea base vulnerable en Git

```
git add .

git status

git commit -m "security: documentar revision estatica y dinamica  con persistencia en TiDB"

git status
```

5.6 Corrige el XSS

Se verifica la correcta configuración de las políticas de seguridad en SecurityConfig.java y el uso de expresiones seguras en las plantillas Thymeleaf (th:text), las cuales aplican escape automático de salida frente a posibles ataques de XSS.

Después de esto se debe de reiniciar el servidor Spring Boot en VS Code antes del retest. Este paso es obligatorio para reflejar los cambios en memoria.

5.7 Retesting

- **Prueba funcional:** Los módulos de gestión del sistema operan correctamente con entradas válidas.
- **Prueba de inyección:** Las consultas a la base de datos en TiDB se procesan de manera segura a través del ORM.
- **ZAP Retest:** Se comprueba la mejora en el estado de seguridad de los endpoints evaluados, manteniendo el registro de las observaciones de cabeceras pendientes de infraestructura.

6. Comparación SAST vs DAST

Criterio	SAST	DAST
Objeto	Código fuente Java, configuraciones y plantillas Thymeleaf.	Aplicación Spring Boot en ejecución (`localhost:8080`)
Necesita ejecutar app	No	Sí, requiere que la aplicación esté corriendo conectada a TiDB/MySQL.
Perspectiva	Análisis interno de la estructura del código y dependencias (pom.xml).	Perspectiva externa sobre las respuestas del servidor (interacción HTTP con vistas y endpoints)
Evidencia del lab	Corrección de mapeos de identidad y validaciones en servicios	Cabeceras HTTP, gestión de sesiones y protección en formularios
Fortaleza	Identifica fallos estructurales en código y dependencias	Valida el comportamiento efectivo del sistema expuesto ante peticiones de red reales.
Límite	No detecta errores de infraestructura o despliegue en runtime	No examina directamente el código interno de los repositorios JPA

7. Evidencia y entrega en GitHub

Genera el reporte en `Docs/security/Security_Analysis.md` incluyendo objetivos, análisis humano, comandos SAST, ajustes en `AdminService.java`, escaneos ZAP y conclusiones.

```
git status

git add Docs/security/

git commit -m "docs: actualizar analisis de seguridad para SecureCampus-preliminar"

git push

git status
```

8. Reflexión

1. ¿Por qué 0 findings en SAST no equivale a aplicación segura?

Porque las herramientas estáticas evalúan patrones de código y sintaxis, pero no pueden validar la correcta aplicación de reglas de negocio complejas ni restricciones de control de acceso basado en roles.

2. ¿Por qué un WARN de ZAP debe validarse antes de declararlo vulnerabilidad?

Porque ciertas advertencias sobre cabeceras pueden estar mitigadas o gestionadas de forma centralizada mediante filtros personalizados en Spring Security o proxies inversos en la infraestructura.

3. ¿Qué diferencia observaste entre baseline y active scan?

El *baseline* realiza una exploración pasiva y rápida, mientras que el *active scan* interactúa de manera activa enviando peticiones de prueba hacia los controladores y formularios de la aplicación.

4. ¿Por qué 200 OK no descarta una vulnerabilidad?

Porque un código de estado exitoso solo confirma que el servidor procesó la solicitud HTTP, pero el contenido devuelto puede carecer de protecciones adecuadas o exponer datos sensibles.

5. ¿Qué aprendiste del hecho de tener que reiniciar Flask antes del retest?

Que tras modificar el código fuente en Java o las plantillas HTML, es indispensable reiniciar el proceso del servidor en VS Code para que el runtime cargue la versión más reciente.

6. ¿Qué problema de autorización podría seguir existiendo aunque SAST y DAST no lo reporten?

Fallos lógicos de control de acceso a nivel de objeto o ruta, permitiendo que usuarios con privilegios limitados accedan a secciones exclusivas de administración.

9. Criterios de aceptación y rúbrica

Criterio	Puntos
Ejecución e interpretación SAST	20
Corrección SQL + reanálisis	15
Baseline y Active Scan DAST	20

Validación/corrección/retesting XSS	25
Comparación SAST vs DAST	10
Evidencia Git/GitHub y reflexión	10
TOTAL	100

Cierre

SAST y DAST aportan evidencias diferentes y complementarias. Ninguna herramienta sustituye requisitos de seguridad, revisión humana, pruebas negativas ni validación de autorización y lógica de negocio.